

Aula 6 - Testes Unitários com JUnit 5

Duração: 2 horas

Objetivos:

- Compreender os fundamentos de testes automatizados
- Dominar anotações do JUnit 5
- Implementar asserções e testes parametrizados

Parte Teórica (40 min)

1. Por que Testes Automatizados?

Benefício	Exemplo Prático
Detecção precoce	Encontrar bug antes do deploy
Documentação viva	Testes explicam o comportamento
Refatoração segura	Alterar código sem quebrar funcionalidades

2. Anatomia de um Teste JUnit 5

```
java

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    @BeforeAll
    static void setup() { /* Inicialização global */ }

    @BeforeEach
    void init() { /* Pré-teste */ }

    @Test
    @DisplayName("Soma de 2 + 2 deve ser 4")
    void testSoma() {
        assertEquals(4, Calculadora.somar(2, 2));
    }

    @AfterEach
```

```
void tearDown() { /* Pós-teste */ }  
}
```

3. Anotações Essenciais

Anotação	Descrição
@Test	Marca um método como teste
@BeforeEach	Executa antes de cada teste
@DisplayName	Nome amigável para o teste
@Disabled	Desabilita um teste temporariamente

4. Asserções Comuns

Método	Exemplo de Uso
assertEquals()	assertEquals(4, resultado)
assertTrue()	assertTrue(lista.isEmpty())
assertThrows()	assertThrows(DivisaoPorZeroException.class, () -> calc.dividir(1, 0))

Parte Prática (1h20min)

Atividade 1: Primeiros Testes Unitários

Cenário: Testar uma classe `Calculadora` com:

- `somar(a, b)`
- `dividir(a, b)` → Deve lançar `DivisaoPorZeroException`

Passo a passo:

1. Crie a classe `Calculadora`:

```
java
```

```

public class Calculadora {
    public static int somar(int a, int b) {
        return a + b;
    }

    public static double dividir(int a, int b) {
        if (b == 0) throw new DivisaoPorZeroException();
        return (double) a / b;
    }
}

```

2. Escreva testes em CalculadoraTest:

```

java

@Test
@DisplayName("Divisão por zero deve lançar exceção")
void testDivisaoPorZero() {
    assertThrows(DivisaoPorZeroException.class, () -> Calculadora.dividir(1, 0));
}

```

Exercício:

- Adicione testes para `somar` com números negativos
- Teste o limite com `Integer.MAX_VALUE`

Atividade 2: Testes Parametrizados

Objetivo: Testar múltiplos inputs sem repetição de código.

1. Adicione dependência ao `pom.xml`:

```

xml

<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <version>5.9.0</version>
    <scope>test</scope>
</dependency>

```

2. Use `@ParameterizedTest` e `@MethodSource`:

```

java

```

```

@ParameterizedTest
@MethodSource("geradorDeTestes")
void testSomaComParametros(int a, int b, int esperado) {
    assertEquals(esperado, Calculadora.somar(a, b));
}

private static Stream<Arguments> geradorDeTestes() {
    return Stream.of(
        Arguments.of(1, 1, 2),
        Arguments.of(-1, -1, -2),
        Arguments.of(0, 5, 5)
    );
}

```

Exercício:

- Crie testes parametrizados para `dividir` com casos como `(4, 2, 2)` e `(9, 4, 2.25)`

Atividade 3: Testes de Vida Real

Cenário: Testar um serviço `UsuarioService` com:

- `validarSenha(String senha)` → deve retornar `false` para senhas curtas

Código para testar:

```

java

public class UsuarioService {
    public boolean validarSenha(String senha) {
        return senha != null && senha.length() >= 8;
    }
}

```

Desafio:

1. Escreva testes cobrindo:

- o Senha `null`
- o Senha com 7 caracteres
- o Senha com 8 caracteres

2. Use `@Nested` para organizar os testes:

```

java

```

```

@Nested
class ValidacaoSenhaTest {
    @Test
    void senhaNula() {
        assertFalse(service.validarSenha(null));
    }
    // ...
}

```

Desafio Avançado (20 min)

Refatore o teste abaixo usando:

1. `@ParameterizedTest`
2. Asserções customizadas (`assertAll`)

```

java

@Test
void testCalculadora() {
    assertEquals(3, Calculadora.somar(1, 2));
    assertEquals(-1, Calculadora.somar(1, -2));
    assertEquals(0, Calculadora.somar(0, 0));
}

```

Solução sugerida:

```

java

@ParameterizedTest
@CsvSource({"1, 2, 3", "1, -2, -1", "0, 0, 0"})
void testSomaParametrizada(int a, int b, int esperado) {
    assertAll(
        () -> assertEquals(esperado, Calculadora.somar(a, b)),
        () -> assertTrue(a + b == esperado) // Validação adicional
    );
}

```

Encerramento

- **Discussão:** Quando usar `@ParameterizedTest` vs testes individuais?
- **Próxima aula:** Mocking e Testes de Integração (Aula 7)

Tarefa:

- Escrever testes para uma classe `StringUtils` com métodos como `isEmpty()` e `invertString()`

Recursos:

- [JUnit 5 User Guide](#)
- [AssertJ para asserções fluentes](#)

Prontos para transformar testes em uma superpotência? Dúvidas? É só perguntar! 🚀💡